LP CLI — Architecture Design REVIEW

Authors: Angel + Tigs
Date opened: 2026-05-13 (night-shift design session)
Last revised: 2026-05-13 — YAML-as-source pivot + A/B/C/D answers locked
Source MD: `docs/design/lp-cli-architecture.md`

1 TL;DR

Build a declarative, idiot-proof CLI for managing La Piazza's operational state. **The secrets rotation tool is the first concrete build** — but the architecture must gene URL routing changes, feature flags, anything the platform exposes that needs to change at runtime.

Pattern: **Resource + Provider + Test, driven by YAML.**

```
declarative YAML ──→ CLI (interactive, paranoid) ──→ apply changes ──→ verify
                                  │
                                  └──→ eventually wrapped as REST API
                                       (same logic, different surface)
```

✅ I reviewed section 1

same logic keep it simple to get the idea of routes URLs for each CLI command maybe think REST API first but skip CLI .. tig tips .. this is a neew tool and we code make it right fir yagni

2 ★ Design Pivot — YAML is source, env is generated output

> **Original design:** the tool edits `borrowhood.env` in place. Env file is BOTH source of truth and runtime config.

> **Pivoted design:** the env file is a **generated output**, not a source of truth. The source is a YAML file (per environment) that lives OFF the git repo. The tool reads YAML

Why the pivot is better

- Single source of truth (the YAML). No drift between human-edited file and runtime config.
- Architecturally impossible to commit secrets to git (env files are generated, gitignored; YAML lives outside repo).
- Per-environment files (`prod.yaml` , `staging.yaml` , future `preprod.yaml` , `uat.yaml`) are the same shape.
- Rollback is symmetric: `prod.previous.yaml` always exists as the last working version.
- Diff-able and dry-runnable.

✅ I reviewed section 2

so for testing we do a stage env and a list and the names like DEV or dv and create a naming convention for expected vatiables like later we have a POS running and we need the fragemented names

3 Why this isn't overkill

Industry has bigger off-the-shelf options. Honest map:

Tool	What it does	Right f
HashiCorp Vault	Centralized secret store, audit, rotation	Overkil
Mozilla sops	Encrypts YAML at rest with age/PGP keys	Easy v2
Doppler / Infisical	SaaS secret manager	$10-30
AWS Secrets Manager	Cloud-native	We're c
k8s Secrets + ESO	Declarative k8s pattern	We use
Manual nano edits	Where solo-dev mistakes live	What w
Our LP CLI	**Tiny YAML-driven CLI, audit, dry-run**	**Right-s**

Verdict: not reinventing the wheel — building the right size wheel for our scale. v2 with sops/age gets us to "production-grade for small team" without throwing aw

✅ I reviewed section 3

but we have pos and the camper and tour and isoto apps and langauge apps and musiic players etc i have tons of apps beyonf la piazza and thats why i want this IMHO

4 The 3 Abstractions: Resource + Provider + Test

Resource

A thing managed declaratively. v1 implements only `secret`. Future: `ai_model`, `route`, `feature_flag`, `quota`.

Provider

Code that knows how to read, write, test, and rotate a specific resource type. One provider per resource type.

```
class SecretProvider:
    def current(name) -> str         # read current value
    def apply(name, new_value)       # set new value
    def test(name) -> bool           # verify it works
    def history(name) -> list        # rotation log
```

Test

Small functions that verify state. Decoupled from Providers so they can be reused.

```
def test_asyncpg_connect_docker_network(env_file: Path) -> bool
def test_oidc_discovery_returns_200(realm_url: str) -> bool
def test_telegram_bot_getme(token: str) -> bool
```

✅ I reviewed section 4

how are we tying this into the keycloak realms and naming conventions and maybe time to think about real prod system wil only real email address and real people and delete o

5 The YAML Schema (post-pivot)

Location: NOT in the git repo. Lives at `/opt/helixnet/secrets/<env>.yaml` on Hetzner, chmod 700 on directory, 600 on each file, owned by root.

One YAML per environment. Rollback pair: before any apply, current snapshots to `<env>.previous.yaml`.

```
# /opt/helixnet/secrets/prod.yaml
version: 1
env_name: prod
generated_env_file: /opt/helixnet/hetzner/lapiazza.env
app_container: borrowhood
hetzner_host: 46.62.138.218

secrets:
  - id: flask-secret-key
    env_var: BH_SECRET_KEY
    value: <actual value>
    rotation:
      sources: [generate, paste]
      generator: openssl_rand_base64_32
      test: app_healthz

  - id: db-password
    env_var: BH_DATABASE_URL
    composite:
      template: "postgresql+asyncpg://{user}:{password}@postgres:5432/{database}"
      parts:
        user: lapiazza_app
        password: <hex value>
        database: borrowhood
    rotation:
      sources: [generate, paste]
      generator: openssl_rand_hex_32
      apply_extra: [alter_user]
      db_admin_role: helix_user
      test: asyncpg_connect_docker_network

  # ... 7 more secret entries
```

✅ **I reviewed section 5**

we need the next rotation dates or some date and time info - like last changed or something know what i am saying

6 CLI Surface

Modes

- `lp-rotate` — fully interactive (menu walk)
- `lp-rotate apply --env prod --secret db-password` — single-secret targeted
- `lp-rotate apply --env prod --dry-run` — show diff, no changes
- `lp-rotate list --env prod` — show all secrets with last-rotated dates
- `lp-rotate status` — overdue/incomplete summary
- `lp-rotate rollback --env prod` — swap previous → current, redeploy
- `lp-rotate validate --env prod` — schema check, no changes

✅ **I reviewed section 6**

LPR apply or lpr rollback but lpr shairting cli name rest is great

7 Directory Structure

```
# OFF-REPO (server only, chmod 700, root):
/opt/helixnet/secrets/
├── prod.yaml                    ← source of truth
├── prod.previous.yaml           ← rollback target
├── staging.yaml
└── staging.previous.yaml

# IN-REPO (the tool):
helixnet/scripts/
├── lp_rotate.py                 ← Typer CLI entry
└── lp/
    ├── cli.py
    ├── manifest.py
    ├── providers/secret_provider.py
    ├── recipes/{edit_env,alter_user,restart_container}.py
    ├── tests/{asyncpg,oidc,telegram,paypal,resend,ollama,healthz}.py
    ├── generators.py
    ├── audit.py
    └── ssh.py

# GENERATED (server, gitignored):
/opt/helixnet/hetzner/
└── lapiazza.env                 ← from prod.yaml
```

> **Rule:** nothing in `/opt/helixnet/secrets/` ever touches git. Nothing in git ever contains real secret values.

✅ **I reviewed section 7**

look looks ok but example of example for stg or UAT or test env for exampples # GENERATED (server, gitignored):
/opt/helixnet/hetzner/
└── lapiazza.env ← from prod.yaml

8 Dependencies (Python only — per CLAUDE rule #11)

- `typer[all]` ≥ 0.12 — CLI framework
- `pydantic` ≥ 2.7 — schema validation
- `PyYAML` ≥ 6 — YAML parser
- `asyncpg` ≥ 0.29 — DB test
- `httpx` ≥ 0.27 — OIDC/Telegram/PayPal/Resend tests

SSH wrapper: shell out to `ssh root@host`. No paramiko. (Per A/B/C/D Q1.)

Python 3.11+. ~6 small libraries. FastAPI-adjacent stack.

✅ **I reviewed section 8**

Notes on this section...

9 Audit Log Format

`~/.lp-rotation-log.jsonl` — one JSON per line, append-only:

```
{"ts":"2026-05-13T08:14:22Z","env":"prod","secret_id":"db-password","actor":"angel","source":"paste","steps":["edit_env","pretest","a
```

Query examples: "when did we last rotate X?" → grep + tail. "Anything failed lately?" → grep failed. Five lines of code, immense future value.

✅ **I reviewed section 9**

does this null out my concerns that we have not dates in the yamls to identifey edit times and logging details of changes and diffs etc

10 Safety Mechanisms (Paranoia in Code, Not Discipline)

1. Backup before any state-changing step (timestamped)
2. Pre-test BEFORE state change (idempotent if already correct)
3. Re-test AFTER state change (rotation only "complete" if test passes)
4. Auto-rollback on failure (restore backup, restart with old creds)
5. Hidden input (`read -s` style) for password paste
6. Compare-paste enforcement: prompt twice, strcmp, mismatch = abort
7. KeePass as single source of truth (tool never tries to store)
8. Append-only audit log of every action including failures

✅ **I reviewed section 10**

eventally have the v2 keepass api working to assit and make it hands free and just done or error etc occurs maybe v2.4
or v3 or v4 unless its super easy would and should be the end goal

11 v1 Milestone Scope

What gets built first (only what's needed for 9 prod secrets):

1. CLI entry + Typer commands
2. Manifest loader + Pydantic schema
3. SecretProvider
4. Recipes: edit_env (generate_env_from_yaml), restart_container, alter_user
5. Tests: asyncpg_docker_network, oidc_discovery, telegram_getme, paypal_sandbox_oauth, resend_api_me, ollama_models_list, app_healthz
6. Generators: openssl_rand_hex_32, openssl_rand_base64_32
7. Audit log writer
8. Manifest YAML seeded from current prod state

Estimated: ~680 LOC across `scripts/lp/` . Single focused session ~10h.

✅ **I reviewed section 11**

since we are askinf for a pw should we not scrable it so what you see in the yaml is not the actual pw but a derived value of kinf of a algorithim is applied and the actaual kex valu

12 v2+ Deferred (Architecture Allows But Not Built)

- Encryption at rest for YAMLs (sops / age)
- Other resource types (ai_model, route, feature_flag)
- REST API wrapper for the same Provider classes
- KC admin REST API integration for IDP secret updates
- Multi-env diff (`lp-rotate diff prod staging`)
- KeePass backup API
- Non-interactive mode (`--yes` flag)
- Concurrency / locking (v1 trusts solo operator)
- Web UI

✅ **I reviewed section 12**

Multi-env diff (lp-rotate diff prod staging) need it i think simple why not just asking

13 Original 6 Questions — Resolved

Q	Decision
SSH wrapper	**Shell out** to ssh. No paramiko.
YAML vs Python config	**YAML**
Env var override on YAML	**NO** — single source of truth
Dry-run mode	**YES**
Logging + audit log	**YES both**
pytest for safety-critical recipes	**YES**

Post-pivot 4 questions — also resolved

Q	Decision
A. Encryption at rest for YAMLs	**v1 plain at 600 root**. Defer encryption to v2.
B. Always show diff on apply	**YES**, interactive confirm. `--yes` for non-interactive.
C. Staged file (prod.staged.yaml)	**NO**. Direct edit + dry-run is enough.
D. KeePass backup cadence	**Manual** for v1.

✅ **I reviewed section 13**

we need to ask more questions about keepass power features IMHO

14 Extra questions (last-pass before code)

1. **YAML seed/import tool** — first run needs to bootstrap prod.yaml from existing env. Add `lp-rotate import --env prod --from /opt/helixnet/hetzner/b`
2. **Disaster recovery doc** — "if Hetzner dies, cold-start procedure." One-page runbook. *Yes, after v1 code.*
3. **Telegram notify on apply** — wire ops bot to ping you on every rotation. *~15 LOC. Yes.*
4. **Validate-only mode** — `lp-rotate validate --env prod` checks YAML schema without applying. *Yes, ~10 LOC.*
5. **Staging-before-prod soft check** — warn (not block) if no recent staging apply. *Yes, ~20 LOC.*

If you approve all 5, v1 grows by ~75 LOC to ~755 total. Still in one session.

✅ I reviewed section 14

1 yes, 2 yes 3 yes 4 yes 5 yes

15 ★ Ready to Build

All design questions resolved. Architecture locked.

Build order (~10h)

1. Pydantic schema for the YAML
2. Manifest loader
3. Recipe: generate_env_from_yaml
4. Recipe: restart_container
5. CLI: apply --dry-run
6. CLI: apply (full with diff confirmation)
7. Tests: asyncpg_docker_network, app_healthz
8. Recipe: alter_user
9. Remaining provider tests (5)
10. Audit log writer
11. Rollback path
12. pytest unit tests

Acceptance criteria

- `lp-rotate list` shows all secrets
- `--dry-run` shows diff without changes
- `apply` works end-to-end with auto-rollback on test failure
- `rollback` restores from previous YAML
- pytest passes
- First real use: finishes the stuck DB password rotation from 2026-05-12

> **Sign off below to greenlight v1 build.**

✅ I reviewed section 15

lets redisplay and review html v2 draft 2 with the confirmations like stage two or three this is first review we need at least 3 to 4 imho tigs

Master Notes

draft 2 rready for prep and stage 2 in the process of deign reviews .. andiamo